Git, GitHub & Environment Setup Guide

vibecoded-pokemon-engine - step-by-step reference for cloning, building, testing, and working with Git branches on this project.

Repository: https://github.com/WillMoose23/vibecoded-pokemon-engine.git

Part 1 - Definitions (Glossary)

Git
Version control on your computer. Tracks file changes, branches, and history.

GitHub
Cloud hosting for Git repositories. The remote named origin points here.

Repository (repo)
The project folder plus its full Git history.

Remote
A named link to a copy of the repo on another machine. origin is GitHub.

Branch
An independent line of work. main and development can differ until merged.

Commit
A saved snapshot of your changes with a message describing why.

Push
Upload local commits to GitHub (origin).

Pull
Download commits from GitHub and merge them into your current branch.

Fetch
Download remote updates without merging yet. Safe preview of what changed.

Merge
Combine another branch into your current branch (or on GitHub via Pull Request).

Pull Request (PR)
GitHub UI to review and merge one branch into another (e.g. development into main).

Staging (git add)
Mark which changed files will be included in the next commit.

Working tree
Your live files on disk. Uncommitted edits live here.

HEAD
The commit your current branch points to - your checked-out snapshot.

Tracking branch
Local branch linked to a remote branch (e.g. development tracks origin/development).

Stash
Temporarily shelve uncommitted changes so you can switch branches cleanly.

Reset
Move HEAD (and optionally files) to an older commit. Can discard local work.

Force push
Overwrite remote history. Dangerous on shared branches - avoid on main/development.

Merge conflict
Git cannot auto-combine two edits to the same lines. You must resolve manually.

Part 2 - Branch Model for This Project

Use these branches consistently:

- main - stable/production baseline. Merge-only from development via GitHub PR.
- development - daily integration branch. Commit and push work here.
- development-local-ai - optional sandbox for local AI / experimental work. Safe to reset without affecting main or development.

Typical flow:

```
1. Work on development (or development-local-ai for experiments)
2. Commit + push to your branch
3. Open a Pull Request: development into main on GitHub
4. Review and merge when ready
```

Never push directly to main unless you explicitly intend to and understand the risk.

Part 3 - Fresh Environment Setup (After Clone)

3.1 Prerequisites (macOS)

- Xcode Command Line Tools: xcode-select --install
- Homebrew: https://brew.sh
- Git (usually included with Xcode CLT)
- Python 3.9+ (macOS includes python3)

3.2 Clone the repository

```
cd ~/Desktop   # or your projects folder
git clone https://github.com/WillMoose23/vibecoded-pokemon-engine.git
cd vibecoded-pokemon-engine
git fetch origin
git checkout development
git pull origin development --no-rebase
```

3.3 Install build dependencies (C++ game + SDL)

```
brew install sdl2 sdl2_ttf sdl2_image sdl2_mixer pkg-config
```

sdl2_mixer is optional; the Makefile enables audio when the library is present.

3.4 Install Python tools (map editor)

```
python3 -m pip install --user pygame
# Optional: generate this PDF
python3 -m pip install --user fpdf2
```

3.5 Build and run the game

```
make
make run
```

3.6 Run tests (required before pushing)

```
make test
python3 -m unittest discover -s tests -q
```

3.7 Run the map editor (optional)

```
python3 tools/map_editor.py
```

Run from the repository root. Requires pygame.

3.8 Optional: GitHub CLI

```
brew install gh
gh auth login
```

Use gh pr create to open Pull Requests from the terminal.

Part 4 - Daily Git Commands

4.1 Check status (always start here)

```
cd /path/to/vibecoded-pokemon-engine
git status
git branch -vv
git remote -v
```

4.2 Pull latest (default: development)

```
git fetch origin
git checkout development
git pull origin development --no-rebase
```

If you have uncommitted changes, commit or git stash before pulling.

4.3 Create a new branch

```
git checkout development
git pull origin development --no-rebase
git checkout -b my-feature-branch
```

Push the new branch the first time:

```
git push -u origin my-feature-branch
```

4.4 Switch branches

```
git checkout development
# or
git switch development
```

4.5 Stage and commit

```
git add path/to/file1 path/to/file2
git status
git commit -m "Short reason for the change in 1-2 sentences."
```

Stage everything (use carefully):

```
git add -A
```

4.6 Push

```
git checkout development
git push origin development
```

First push on a new branch:

```
git push -u origin branch-name
```

4.7 Stash uncommitted work

```
git stash push -m "wip before pull"
git pull origin development --no-rebase
git stash pop
```

Part 5 - Merge & Pull Requests

5.1 Merge on GitHub (recommended for main)

1. Push development to origin.
2. Open: https://github.com/WillMoose23/vibecoded-pokemon-engine/compare/main...development
3. Create Pull Request, review, merge.

5.2 Merge locally (use with care)

```
git checkout development
git pull origin development --no-rebase
git checkout main
git pull origin main --no-rebase
git merge development
git push origin main
```

Prefer GitHub PRs for merging into main so you get review and CI checks.

5.3 Create PR with GitHub CLI

```
git push -u origin development
gh pr create --base main --head development \
  --title "Release: summary" \
  --body "What changed and how to test."
```

Part 6 - Reset, Rollback & Recovery

6.1 Discard uncommitted changes in one file

```
git restore path/to/file
```

6.2 Discard ALL uncommitted changes (destructive)

```
git restore .
git clean -fd   # removes untracked files - irreversible
```

6.3 Undo last commit, keep file changes

```
git reset --soft HEAD~1
```

6.4 Move branch back one commit, discard changes (destructive)

```
git reset --hard HEAD~1
```

6.5 Reset sandbox branch to match development

Useful for development-local-ai when experiments went wrong:

```
git fetch origin
git checkout development-local-ai
git reset --hard origin/development
```

This does not affect main or development on GitHub - only your local sandbox branch until you push.

6.6 Revert a commit safely (new commit that undoes)

```
git log --oneline -5
git revert <commit-hash>
git push origin development
```

Part 7 - Troubleshooting

Merge conflicts after pull

1. Git marks conflicted files. Open them and fix <<<<<<< markers.
2. git add <fixed-files>
3. git commit (merge commit) or git merge --continue

Branch behind / ahead

```
git fetch origin
git status   # shows ahead/behind vs origin
```

Wrong branch

```
git checkout development
```

See what changed

```
git diff
git diff --staged
git log -5 --oneline
git log main..development --oneline
```

Part 8 - Pre-Push Checklist (This Project)

Before pushing to development:

- Update docs/source_doc.md and/or docs/tools_doc.md for code changes.
- Log work in docs/tracker.md with accurate STATUS.
- Run: make test && python3 -m unittest discover -s tests -q
- Optional sync: python3 tools/sync_cursor_backup.py
- Never commit secrets (.env, tokens).
- Push to development, not main.

Quick Reference Card

```
PULL:     git fetch && git checkout development && git pull origin development --no-rebase
COMMIT:   git add <files> && git commit -m "message"
PUSH:     git push origin development
BRANCH:   git checkout -b new-branch
MERGE:    GitHub PR development into main
RESET:    git reset --hard origin/development  (sandbox only)
BUILD:    make && make test
```